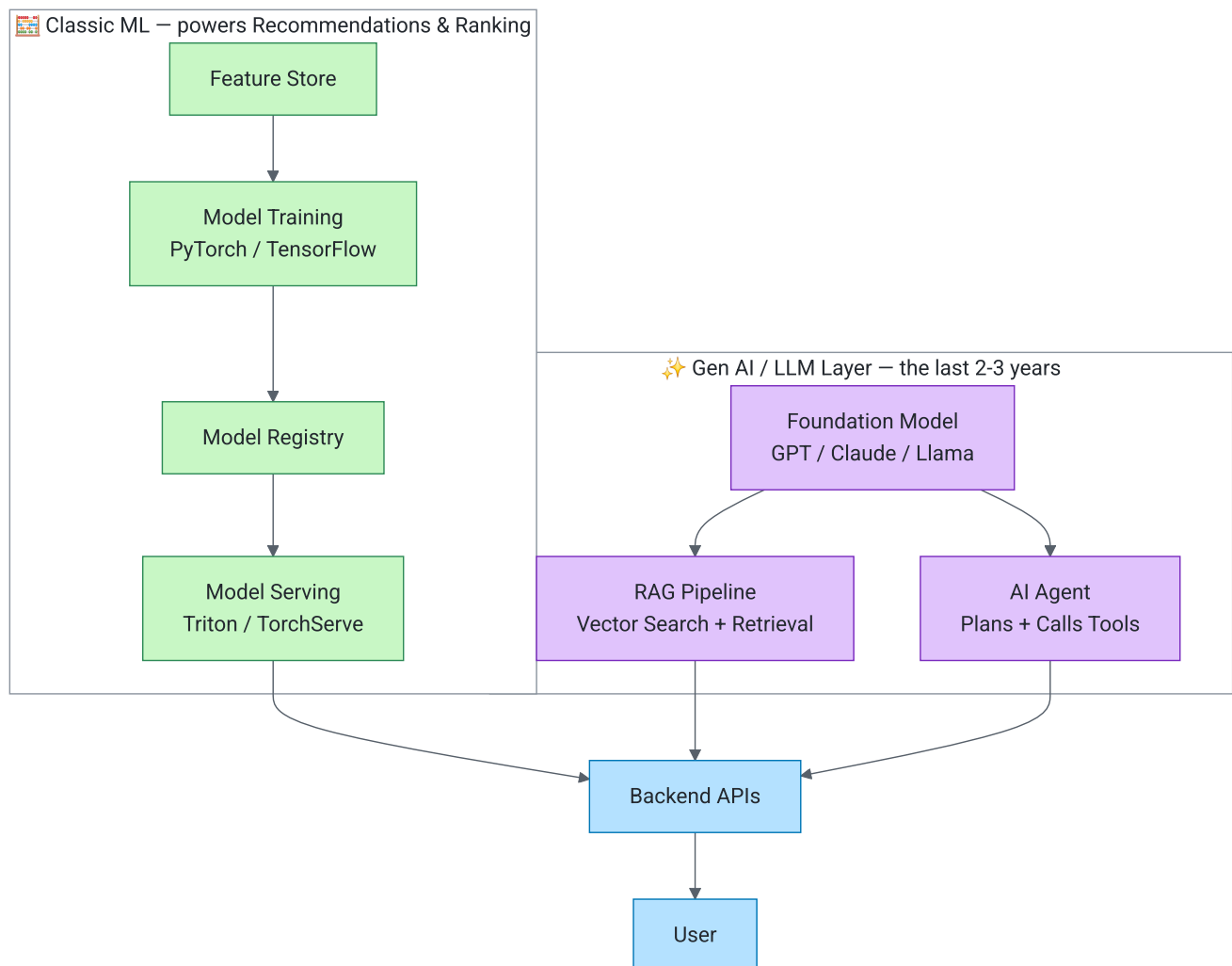


AI/ML Engineering, Zoomed In – Gen AI, LLMs & Agents at YouTube

The earlier doc mentioned one box called "ML Serving." Open it today and you'll find two generations of AI living side by side: the classic ML that's quietly ranked your feed for a decade, and the newer generative/agentive layer that's exploding right now. This is the zoom-in on both – with heavy focus on what's actually new.

1. The Big Picture – Two Eras, One System



The distinction that matters: classic ML *predicts* (what will you click next?). Gen AI *generates* (write a summary, answer a question). Agents *act* (actually go do something — search, filter,

save — on the user's behalf). Three different capabilities, increasingly stacked on top of each other.

2. The Components, One at a Time

Foundation Models — the new "database of knowledge"

Instead of training a model from scratch, teams now call or fine-tune a pre-trained foundation model.

- **Hosted API:** OpenAI GPT, Anthropic Claude, Google Gemini — no infra to manage, pay per token.
- **Self-hosted OSS:** Llama, Mistral, Qwen — full control, needed for data privacy or cost at scale.
- **Why it matters:** the build-vs-buy decision for AI just changed completely — most teams now start with an API call, not a training run.

RAG (Retrieval-Augmented Generation) — grounding the model in *your* data

An LLM alone doesn't know about a video uploaded five minutes ago. RAG fixes this: retrieve relevant facts first, then hand them to the LLM as context before it answers.

- **Tools:** Pinecone, Weaviate, pgvector, FAISS (vector databases) + an embedding model
- **Why it matters:** this is how "Ask AI about this video" features work — the answer is grounded in the actual transcript, not the model's memory.

Fine-tuning / LoRA — specializing a general model

Full fine-tuning is expensive; LoRA (Low-Rank Adaptation) freezes most of the model and trains a small set of extra parameters — cheap enough to do per use-case.

- **Tools:** Hugging Face PEFT, Axolotl
- **Why it matters:** lets a team specialize a general model for, say, moderation tone or a specific content category, without retraining billions of parameters.

LLM Inference & Serving — making generation fast enough to ship

Generating text token-by-token is slow and expensive at scale. Serving frameworks optimize batching, memory, and latency.

- **Tools:** vLLM, TensorRT-LLM, quantization (running smaller-precision versions of the same model)
- **Why it matters:** the difference between a chatbot that feels instant and one that feels laggy usually lives here.

AI Agents — the model that takes action, not just answers

An agent doesn't just respond — it **plans a sequence of steps, calls tools, observes results, and loops** until the goal is done. "Find me a 10-minute pasta video and save it" requires search, filtering, and an action — three steps, not one Q&A.

- **Tools/Frameworks:** LangGraph, CrewAI, AutoGen — for orchestrating multi-step or multi-agent workflows
- **Protocols:** MCP (Model Context Protocol) standardizes how an agent discovers and calls external tools/data sources; A2A (Agent-to-Agent) is the emerging standard for agents talking to *other* agents
- **Why it matters:** this is the newest and fastest-moving layer in the entire AI stack right now.

Agent Memory — short-term vs long-term

An agent's context window is short-term memory (what fits in the current conversation). For anything durable — user preferences, past interactions — agents read/write to an external memory store.

- **Tools:** vector DBs (same ones used for RAG), Redis for session state

Observability & Evals — because "it sounded right" isn't good enough

Unlike classic ML with clean accuracy metrics, LLM output is fuzzy — you need tracing (what did the agent actually do, step by step?) and evals (is this answer actually correct/safe?).

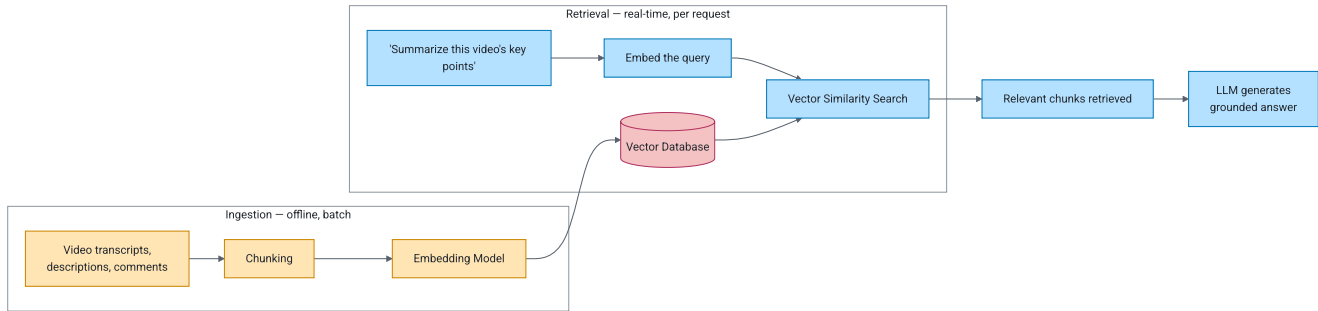
- **Tools:** LangSmith, Langfuse, Arize Phoenix

Guardrails & Safety — the seatbelt

Filtering unsafe outputs, redacting PII, detecting prompt injection attempts before a tool call executes something it shouldn't.

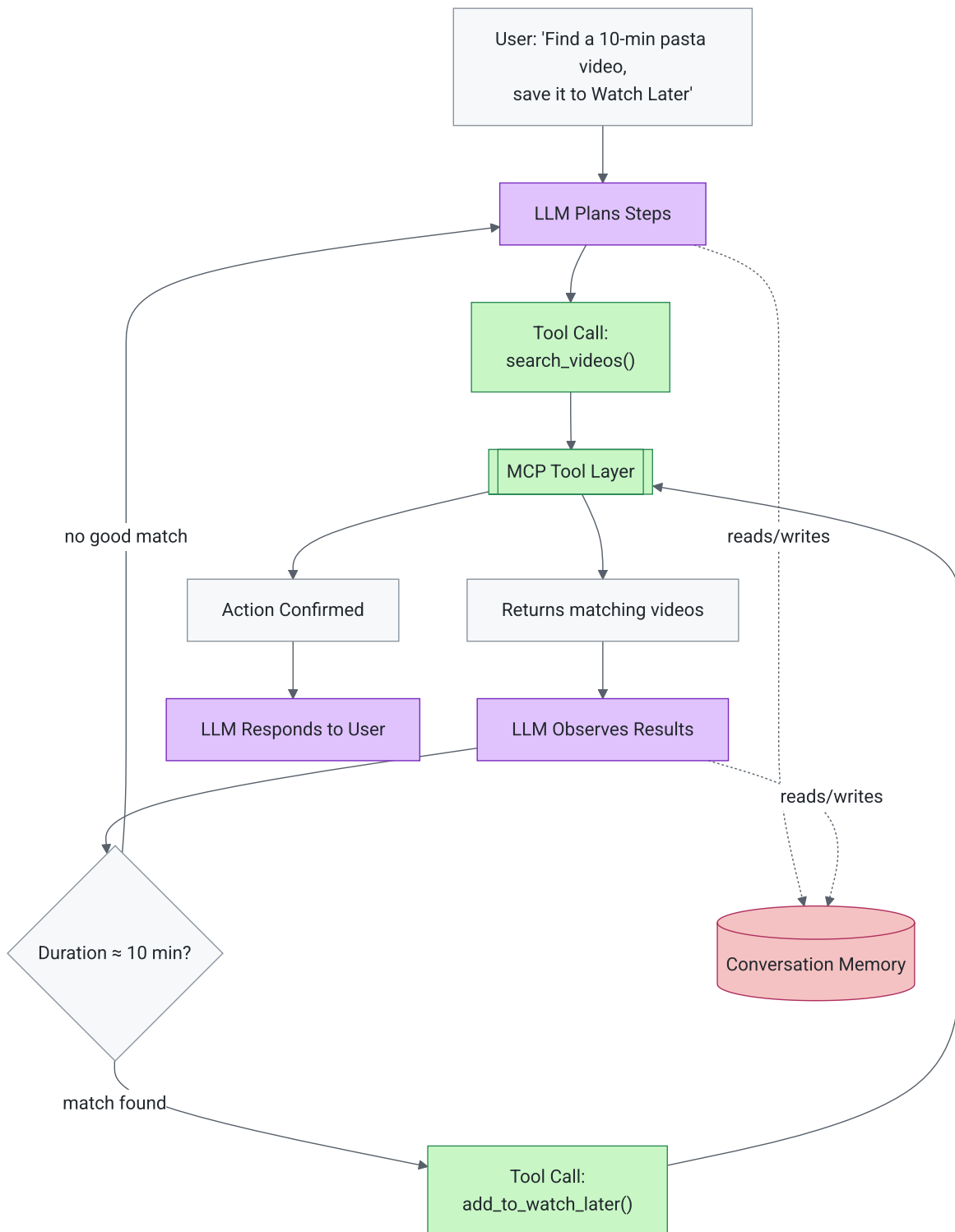
- **Tools:** Guardrails AI, custom moderation models, allow-listing which tools an agent can call

3. Zoom: How RAG Actually Works



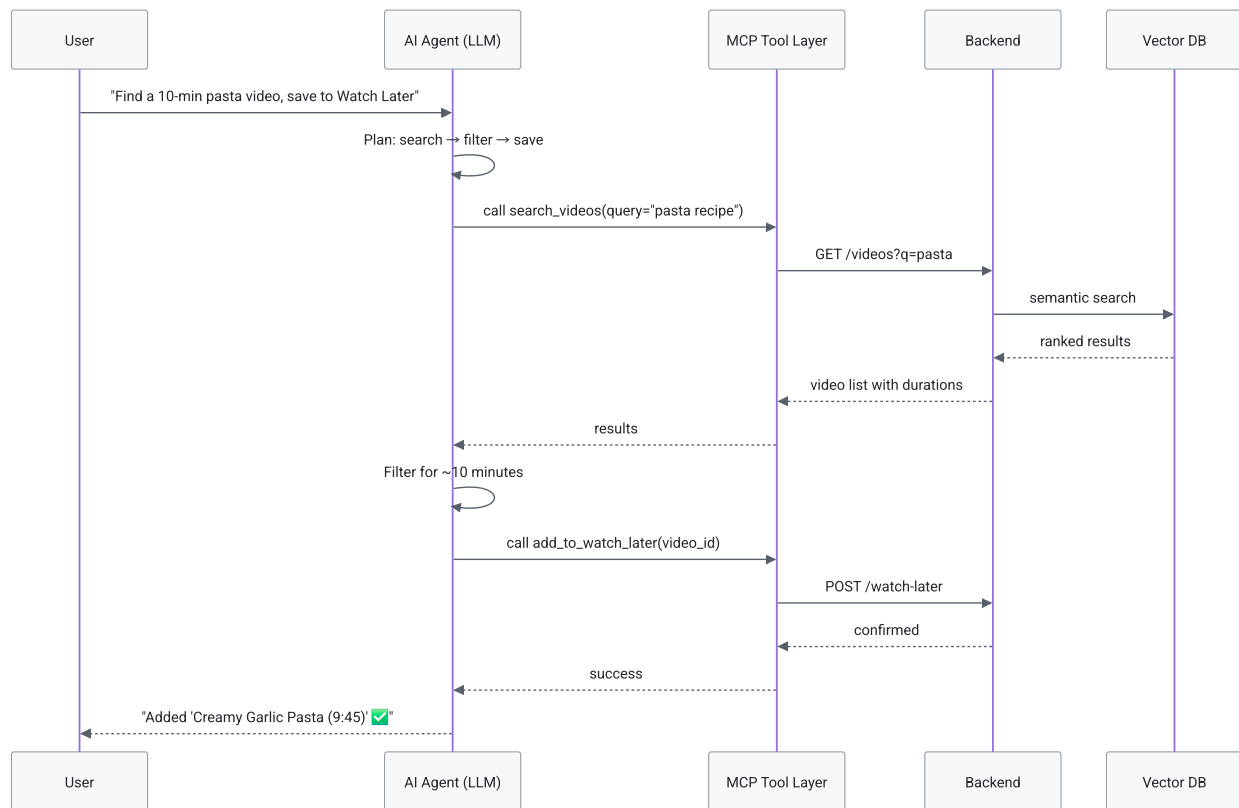
The whole trick: the model isn't guessing from memory — it's reading the actual transcript chunks handed to it, then writing an answer. This is why RAG dramatically cuts down on hallucination for factual, document-grounded questions.

4. Zoom: The AI Agent Loop



This loop — **plan** → **act** → **observe** → **repeat until done** — is the core pattern behind every AI agent, whether it's booking a flight, writing code, or fetching a video. The model isn't one-shotting an answer; it's iterating like a junior engineer would, checking its own work as it goes.

5. Putting It Together — "Find Me a Video and Save It"



Notice this reuses the *exact same backend* from the previous doc — the agent doesn't replace the Video Service or the Database, it sits on top and calls them as tools. Gen AI engineering isn't a separate universe; it's a new client talking to the same backend, through a standardized tool-calling layer.

6. Component Cheat Sheet

Component	Problem it solves	Popular Tools	Note
Foundation Model	General-purpose reasoning/generation	GPT-4, Claude, Llama, Gemini	Build-vs-buy: API first, self-host later
RAG / Vector DB	Grounding answers in real data	Pinecone, pgvector, FAISS	Cuts hallucination on factual questions
Fine-tuning (LoRA)	Specializing a model cheaply	Hugging Face PEFT, Axolotl	Small deltas, not full retrains
LLM Serving	Fast, cheap inference at scale	vLLM, TensorRT-LLM	Batching + quantization =

Component	Problem it solves	Popular Tools	Note
			latency wins
Agent Framework	Multi-step planning & orchestration	LangGraph, CrewAI, AutoGen	Fastest-moving layer in AI right now
Tool-calling Protocol	Standardizing agent ↔ tool access	MCP, function calling	Lets one agent use many tools safely
Agent-to-Agent	Multiple agents collaborating	A2A (emerging)	Still early, worth watching
Observability/Evals	Trusting fuzzy LLM output	LangSmith, Langfuse, Arize Phoenix	Tracing + automated eval scoring
Guardrails	Safety, PII, injection defense	Guardrails AI, custom filters	The seatbelt on autonomous actions

7. Why This Is the Exciting Part Right Now

Classic ML engineering matured over a decade — the patterns are well understood. Gen AI and agents are being invented *in public, right now* — the frameworks, protocols, and best practices you'd learn today (MCP, LangGraph, RAG patterns) barely existed two years ago. For a fresher, that's rare: everyone, senior or junior, is genuinely new at this layer. The gap between "know the concepts" and "have actually built an agent" is smaller here than in almost any other part of the stack — which makes it one of the highest-leverage places to build real, demonstrable projects early in a career.